



Candidate Assessment | Mobile.Developer (Gutendex API)

Thank you for participating in this exercise. This is a standard part of our evaluation process for all candidates, and we're excited to see what you can do.

Please don't hesitate to reach out with questions—we're here to help! Feel free to discuss your approach, bounce ideas off us, and take advantage of any resources we've provided. We want you to enjoy the process and do your best work.

Instructions:

- Write clean, readable code with consistent indentation and clear naming conventions for variables, functions, classes, and methods
- Include comments where they add value and clarify your logic
- You're welcome to use any AI tools like ChatGPT, GitHub Copilot, or similar assistants. We encourage you to leverage these modern tools—they're part of today's development landscape. However, please be prepared to explain any code you use (whether AI-generated or from other sources) and understand it completely.
- If you use AI-generated code, let us know and be ready to discuss how you validated and adapted it for your solution.
- The designs shared in the problem statement are for a responsive app. We expect you to implement the provided designs as a **fully functional mobile application**
- Your solution should be architected for **theming** and **multi-lingual** capability. It is sufficient if the code is easily modifiable to **support different themes or languages—these capabilities do not have to be exposed from the UI.**
- Ensure the layouts remain usable and visually correct in both **portrait and landscape** orientations.
- The submitted code should be production-ready, with appropriate architecture, maintainability, error handling, configuration management, security considerations, and overall code quality.



Candidate Assessment | Mobile.Developer (Gutendex API)

Submission:

- Please email your solutions to the assessment email shared or email them to careers@ignitesol.com
- Please submit your solution as a public Git repository link. Ensure your repository includes a clear README with setup and run instructions.
- Use meaningful, feature-wise Git commits as you progress. Avoid submitting the entire project in a single commit, and use clear, relevant commit messages.
- Include a demo video of the app in the GitHub README.
- Please email the application binary (e.g., APK or equivalent) along with your submission.
- The README must include :
 - Project overview
 - Setup and run instructions
 - Architecture overview
 - Third-party libraries used
 - AI tools used (if any)
 - Assumptions and known limitations
 - Demo video (in both portrait and landscape orientations)

We're rooting for you—let's get started!

Question

The Gutendex API is a hosted instance of the open-source [project](#), which in itself is an extension of data available from [gutenberg.org](#), a repository of freely available books. It contains a single endpoint, with a set of query parameters that allow for filtering the books in several manners.

The hosted instance is available on <https://gutendex.careers.ignitesol.com>. The API specification is provided on the last page of this document.

Description

The app has 2 distinct pages, the designs for which are provided [here](#).

The first page will have a title of the app and a list of buttons labelled by the category/genre of books available.

Clicking on the buttons should transition the user to the next page, which would display the books matching the selected category. The books SHOULD be displayed in an infinite-scrolled list (fetching more books as the user scrolls through the list). If the user enters any text in the search bar, the API MUST be invoked again to search for all the books whose title or author match the given input text, while maintaining the currently selected category/genre filter.

The application should gracefully handle loading, empty, and network error states, and provide appropriate feedback to the user.

The API response contains links for books in different formats. If the user taps on any book the app **must open the web browser** pointing to the book in one of the following formats, the format higher in the list must be preferred, falling back to the next available format (if given format is not available)

- HTML
- PDF
- TXT

If none of the above **formats** are available, the **app must display an alert box** with an error message “No viewable version available”.

Caveats

1. Zip files are NOT viewable files.

(Hint: This issue can be easily worked around, bonus points for figuring it out) 2. API response should only contain the books with covers.

(Hint: ONLY query for books that have a mime_type with images)

Example

If the user taps **Fiction** on the first page, the next page will display a scrollable list of all the books of category/genre **Fiction**.

Thereafter, if the user searches "**Vampire**" in the search box, the list MUST be filtered via the API with ALL books that are **Fiction (Subjects and Bookshelves)** AS WELL AS **Vampire (Title and Authors)** based books.

Upon tapping on the book, the web browser opens with either (in order of priority) the HTML version or the PDF version or the TXT version of the book (or gives an error alert).

The core requirements are expected to be completed in approximately 3 hours.

Below is the API as taken from the GitHub link. It is a working and tested setup.

Any provided designs MUST be followed.

Good to Have (Optional)

The following are optional and will not affect the correctness of your solution but may be considered positively during evaluation:

- UI tests (optional)
- Offline caching
- Pull-to-refresh
- Accessibility considerations
- Performance optimizations
- Smooth animations and polished UI interactions



Candidate Assessment | Mobile.Developer (Gutendex API)

API Specification

Base API URL: <https://gutendex.careers.ignitesol.com>

Lists of Books

Lists of book information in the database are queried using the API at /books (e.g. <https://gutendex.careers.ignitesol.com/books>). Book data will be returned in the JSON format

```
{
  "count": <number>,
  "next": <string or null>,
  "previous": <string or null>,
  "results": <array of Books>
}
```

where results is an array of 0-32 book objects, next and previous are URLs to the next and previous pages of results, and count is the total number of books for the query on all pages combined.

Books are ordered by popularity, determined by their numbers of downloads from Project Gutenberg. The following query parameters can be passed for filtering the books

ids

Use this to list books with Project Gutenberg ID numbers in a given list of numbers. They must be comma-separated positive integers. For example, </books?ids=11,12,13> gives books with ID numbers 11, 12, and 13.

languages

Use this to find books in any of a list of languages. They must be comma-separated, two-character language codes. For example, </books?languages=en> gives books in English, and </books?languages=fr,fi> gives books in either French or Finnish or both.

mime_type

Use this to find books with a given [MIME type](#). Gutendex gives every book with a MIME type *starting with* the value. For example, /books?mime_type=text%2F gives



Candidate Assessment | Mobile.Developer (Gutendex API)

books with types text/html, text/plain; charset=us-ascii, etc.;
and [/books?mime_type=text%2Fhtml](#) gives books with types text/html, text/html;
charset=utf-8, etc.

search

Use this to search author names and book titles with given words. They must be separated by a space (i.e. %20 in URL-encoded format) and are case-insensitive. For example, [/books?search=dickens%20great](#) includes *Great Expectations* by Charles Dickens.

topic

Use this to search for a case-insensitive key-phrase in books' bookshelves or subjects. For example, [/books?topic=children](#) gives books on the "Children's Literature" bookshelf, with the subject "Sick children -- Fiction", and so on.
